



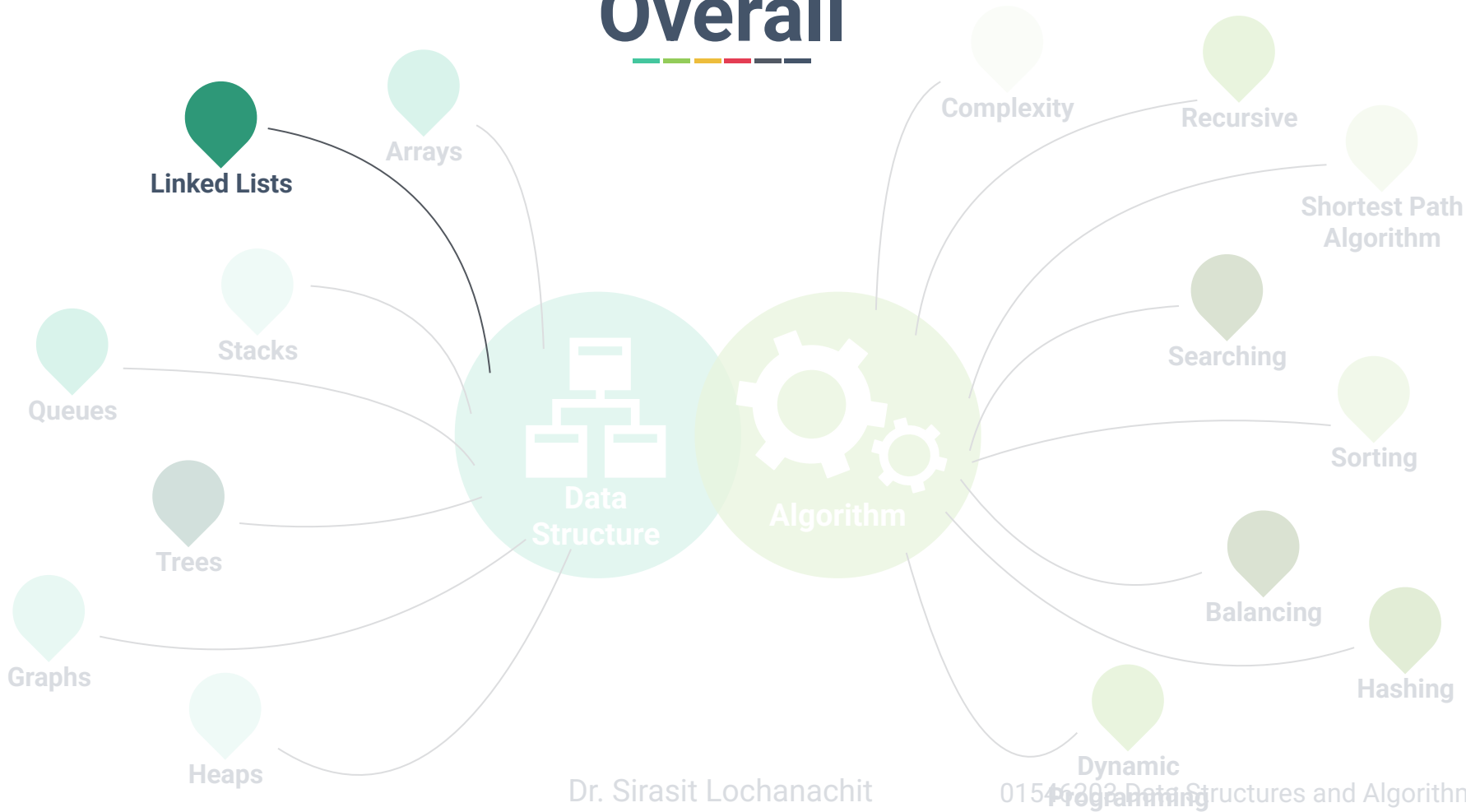
Week 4: Linked Lists



Dr. Sirasit Lochanachit



Overall





Today's Outline

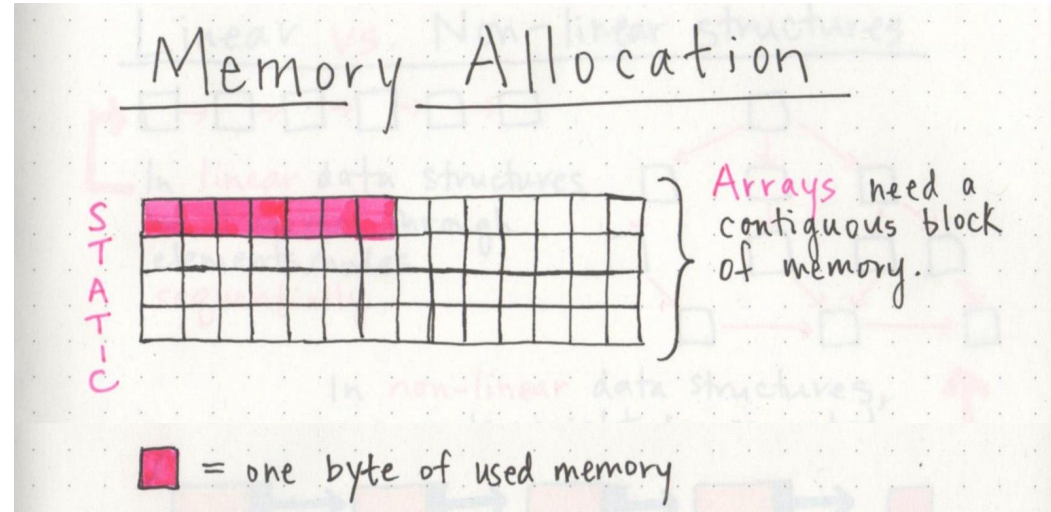
1. What is a Linked List?
2. Singly Linked Lists
 - Traversing
 - Insert a node
 - Delete a node

Previously



Disadvantages of array:

- Length of array has to be **pre-allocated and fixed**, empty space wasted and unable to grow.
- Adding or removing elements between values in the array is expensive - $O(n)$



Linked Lists



To avoid these limitations, an alternative to array is **linked list**.

Array	2	7	8	4	Value
	0	1	2	3	Index

Linked lists



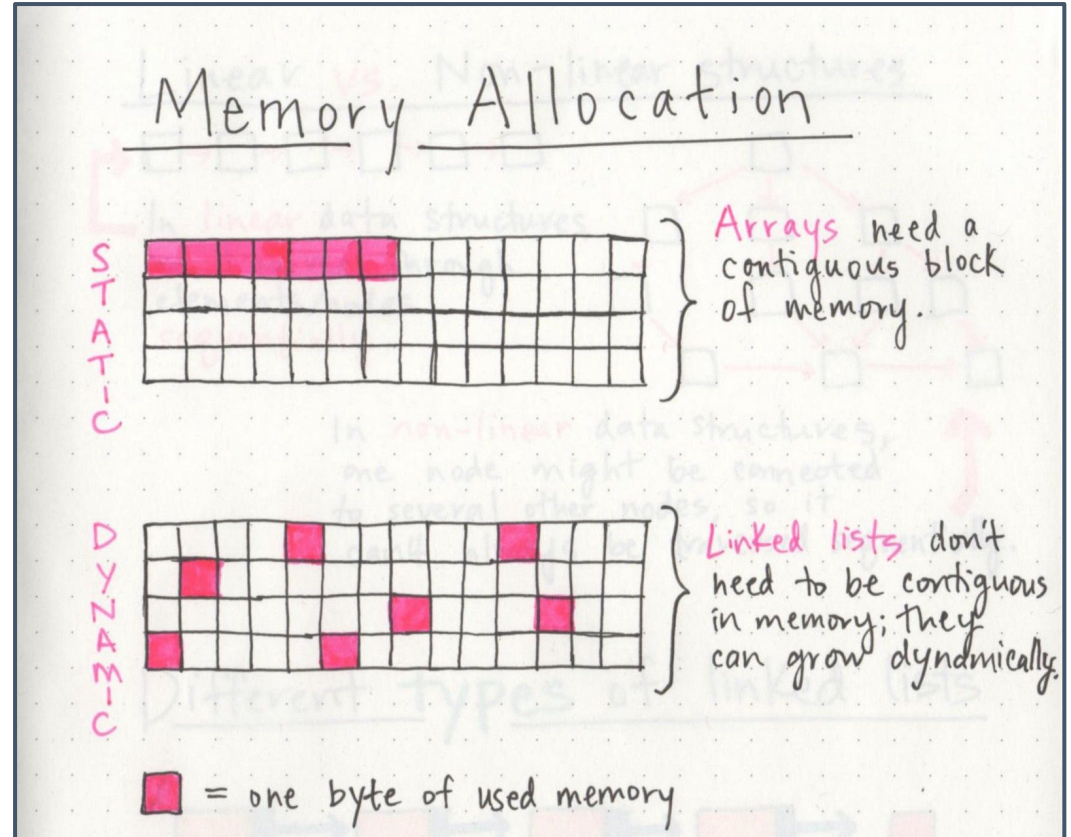
Linked Lists

Static

- Pre-allocated
- Fixed size
 - Unable to grow

Dynamic

- Allocated as needed
- Able to grow





Linked Lists



TYPE

Singly Linked List

Circularly Linked List

Doubly Linked List

Doubly Circularly Linked List

ACTION

Insert

Delete

Search



What is a Linked List?

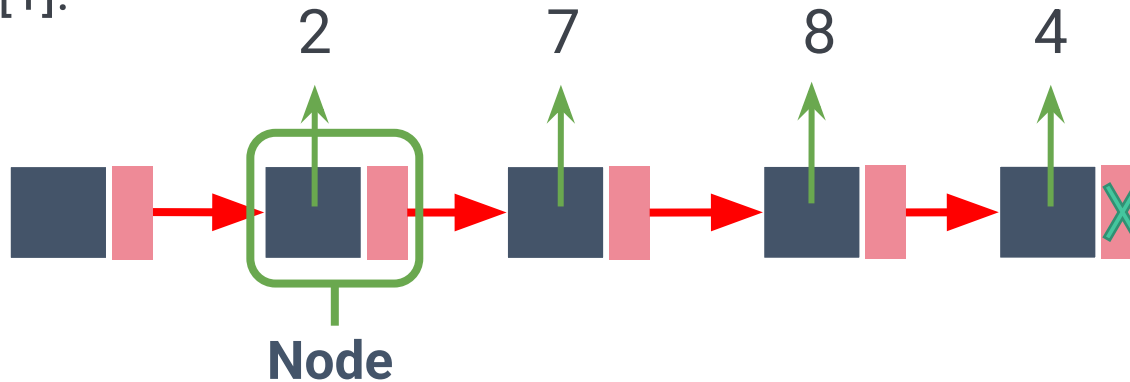


A singly **linked list** is a collection of nodes that form a linear order of a sequence [1].

What is a Linked List?

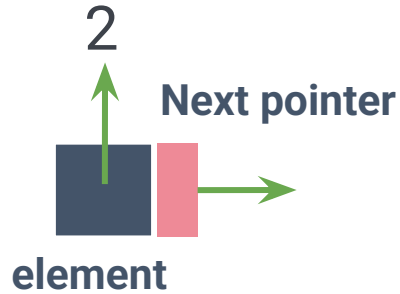


A singly **linked list** is a collection of nodes that form a linear order of a sequence [1].





Linked List Node





Linked List Node Structures





Create a Linked List

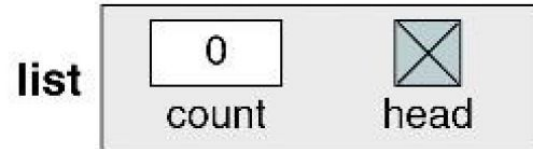


1. Create a sentinel/header/root node

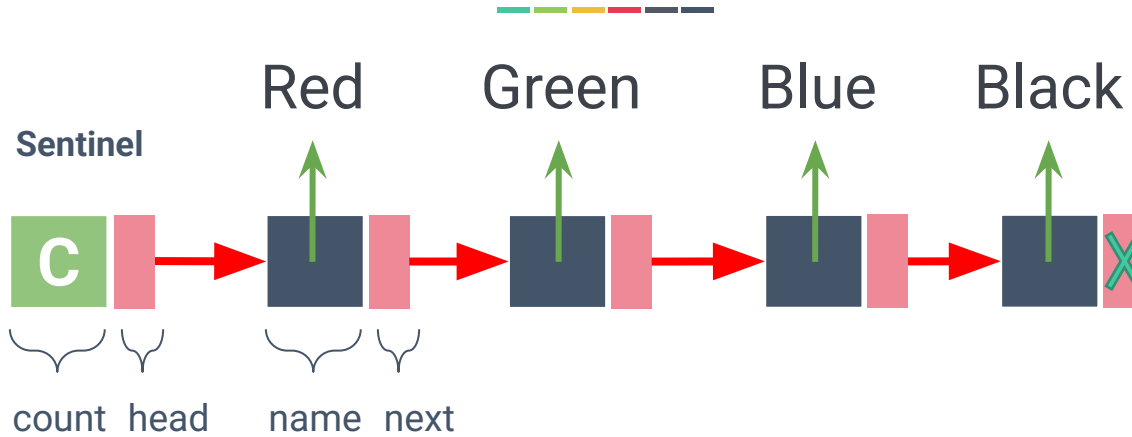
Algorithm createList (list)

1. Allocate a list
2. Set list head to null
3. Set list count to 0

End createList



Singly Linked Lists



Color

String

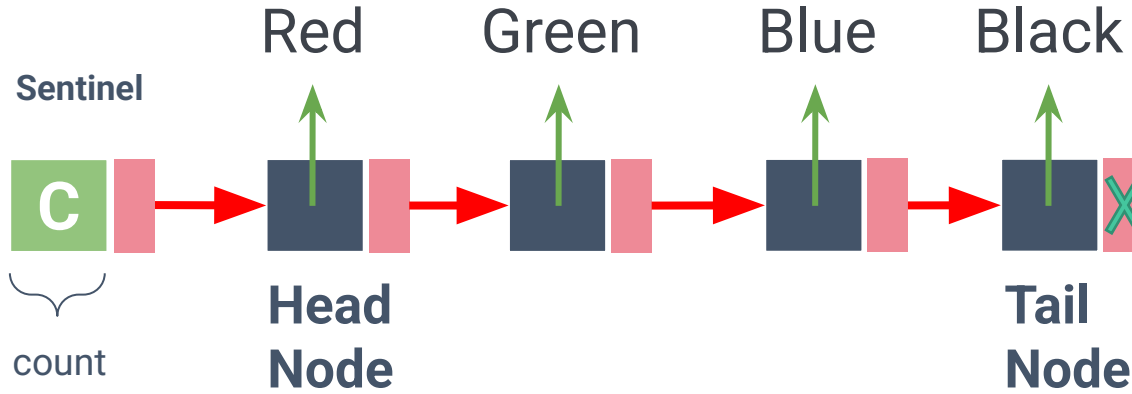
Color next

End Color

name



Singly Linked Lists





Singly Linked Lists





Create a Linked List



2. Create a data/element Node

Algorithm createDataNode (d, p)

Red

colorNew = allocate(Color)

name = d

next = p

return colorNew

End createDataNode



Traversing Singly Linked Lists



Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

Suppose that it takes 1 byte to store an integer.



Traversing Singly Linked Lists

Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

Counter
Next

Header





Traversing Singly Linked Lists

Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

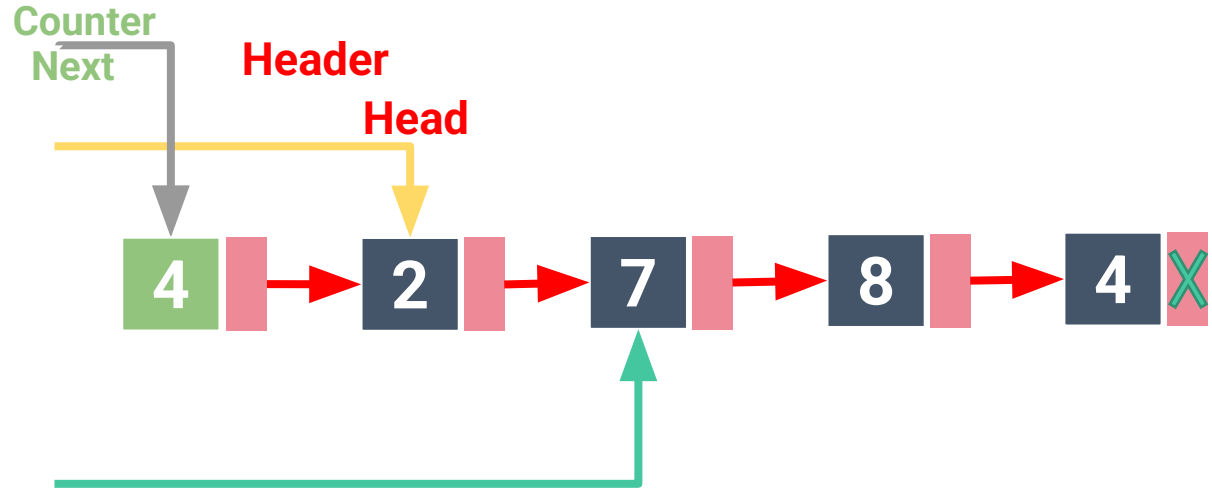




Traversing Singly Linked Lists

Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

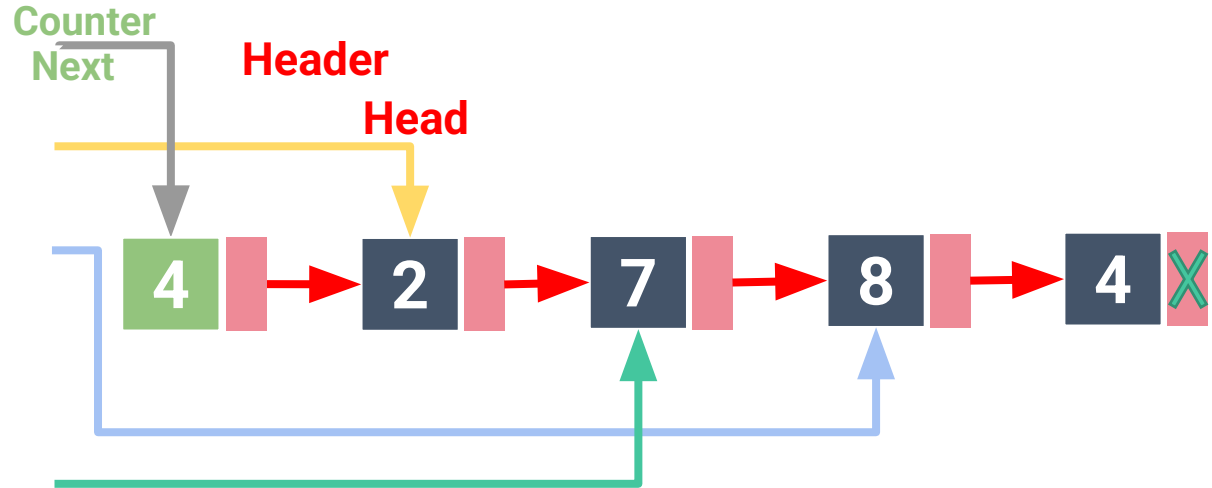




Traversing Singly Linked Lists

Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

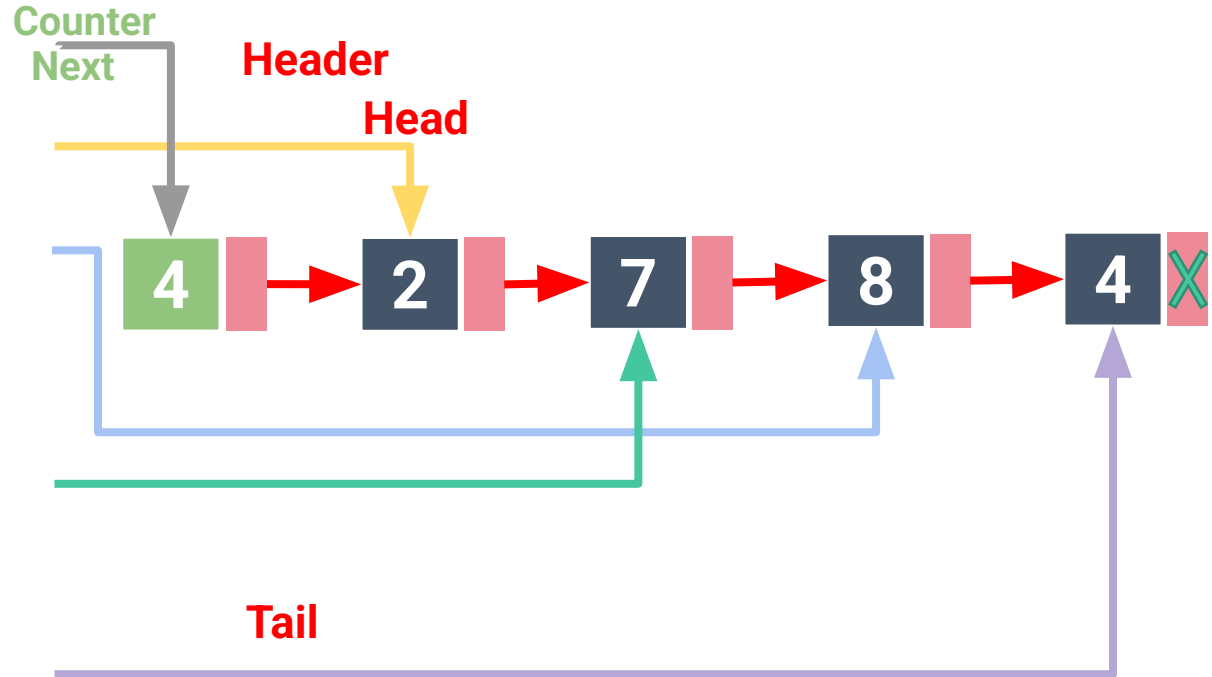




Traversing Singly Linked Lists

Address/
Byte# Value

6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None





Linked List Operations



See the interactive guide on how insertion and deletion works